

Group 30 Final Project Report: Pet Food Dispensing Robot

MTE 121, MTE 100

UNIVERSITY OF
Waterloo



Department of Mechanical and Mechatronics Engineering

A Report Prepared For:
The University of Waterloo

Prepared By:
Waffa Zahralyn: 21188830
Mishal Poldas: 21200898
Maria Jacques: 21212767
Rithanya Jaganathan: 21203678

December 2, 2025

Table of Contents

List of Figures	iii
List of Tables	iv
Acknowledgements	v
Summary	vi
1.0 Introduction	1
1.1 Objective	1
2.0 Scope	2
2.1 Measurements and Detection	2
2.2 Motors	3
2.3 Completion and Shutdown	4
2.4 Changes	4
3.0 Constraints and Criteria	5
3.1 Constraints	5
3.2 Criteria	6
3.3 Criteria and Constraint implementation	7
4.0 Mechanical Design and Implementation	8
4.1 Chassis and Motor Drive	8
4.2 Lever	9
4.3 Sensor Attachment	10
4.4 Container Holder and Slide	10
4.5 Trapdoor Mechanism	11
4.6 Printing and Overall Assembly	12
5.0 Software Design and Implementation	14
5.1 Software Design Overview	14
5.2 Software Design Decisions and Trade-offs	21
5.3 Program Testing and Validation	21
5.4 Significant Problems and Solutions	23
6.0 Verification	24
6.1 Meeting Each Constraint and Criteria	24
6.2 Modified / Unmet Constraints and Criteria	25
7.0 Project Plan	26

7.1 Division of Tasks Among Group Members	26
8.0 Conclusions	27
9.0 Recommendations	28
9.1 Mechanical Design Recommendation	28
9.11 Performance and Reliability: Trapdoor	28
9.12 Suitability For the Home: Materials and Detachability	28
9.2 Software Design Recommendation	29
10.0 Back Matter	30
References	30
Appendix A - C++ Code	31

List of Figures

Figure 1: Original robot self aligning mechanism sketch	5
Figure 2: New bumper lever design.	5
Figure 3: 3D printed base with container on robot.	6
Figure 4: Robot chassis and supports.	8
Figure 5: Lever arm.	9
Figure 6: Lever, pushback support.	9
Figure 7: Section view of CAD, internal incline.	11
Figure 8: 3D printer base.	11
Figure 9: CAD of entire trapdoor mechanism.....	12
Figure 10: CAD of switch.....	12
Figure 11: CAD of trapdoor and slide.	12
Figure 12: Complete 3D print assembled.	13
Figure 13: Section view of slide lip lean in.	13
Figure 14: Flowchart for main()	17
Figure 15: Flowchart for moveHome().	17
Figure 16: Flowchart for startRoutine()..	18
Figure 17: Flowchart for moveUntilBumper()	19
Figure 18: Flowchart for detectColor().....	19
Figure 19: Flowchart for releaseFood()	20
Figure 20: Flowchart for afterDetection()	20
Figure 21: Original Gantt chart for project timeline	26
Figure 22: Two angles of the cardboard prototype for the 3D printed base	26

List of Tables

Table 1: Final Task List for Demonstration and Notes on Changes.....	14
Table 2: Key Program Functions and Implementation Details.....	16
Table 3: Test and Verification Summary for Program Functions.....	22
Table 4: Work Distibution.....	25

Acknowledgements

We would like to acknowledge the valuable support given to us by our MTE 100 and MTE 121 instructors, WEEF tutors, and teaching assistants who have guided us through this project and assisted us when needed. We appreciate the information they have taught to us in order to make this project possible. We would also like to acknowledge VEX Robotics for providing us with the means of robotics components.

Summary

This report covers the design cycle of an autonomous pet-feeding robot. The robot fills a preprogrammed number of bowls with an associated amount of food, aiming to reduce the need to manually fill pet bowls and prevent pet health issues. The system features a VEX based mechanical chassis and 3D printed structures. It navigates through its environment using sensor inputs, which control the conditions of the motors. To account for more than one bowl and the possibility of an obstacle, a for-loop programmed to loop for the number of bowls was utilized to repeat the food dispensing action only when a bowl was detected. Motor encoders allow for the robot to return to original position after task completion. The final robot successfully completed its tasks, however, issues such as misalignment during navigation, jammed food, and food spillage caused by robot movement and vibration made it impractical for real-world use.

1.0 Introduction

Keeping up with consistent feeding routines and diet portioning can become difficult for pet-owners due to demanding schedules. The resulting irregularities in a pet's diet and feeding timings can lead to health issues such as digestive problems and behavioral changes. As households begin to rely on automated processes to improve convenience and save time during daily tasks, this project delves into how automation can be carried onto pet-care as well.

A pet-feeding robot that can dispense food in preprogrammed amounts into a fixed preprogrammed number of bowls can help ease the task of remembering pet food portions and can eliminate the need to manually fill up food bowls at every feeding time. With the touch of a button, the robot can cover these tasks, only requiring the owner to refill the robot's food container at the end of the day.

This report highlights the engineering process, and the mechanical and software design of the pet-feeder robot. It explains the design requirements, constraints and criteria, the methods used during the design process, and decision discussions. The report will present the evaluation of the robot's final demonstration against the project requirements and will explore how the results either met or did not meet the expectations provided.

1.1 Objective

The objective of this project was to create an autonomous pet-feeding robot that could dispense controlled preprogrammed amounts of food into a fixed preprogrammed number of bowls, bypass obstacles within the line-up of bowls, and return back to its starting position. The goal involved designing a structurally sound base using VEX components for the 3D printed structures housing the food container on top. It also required the design of an efficient mechanism to dispense the food which involved the usage of sensor inputs and motor activation.

2.0 Scope

The pet food bowls will be lined up approximately a chosen distance apart and the robot will be situated 3-5 inches beside the line-up of bowls. The robot's routine will be initiated through the press of a button. Its routine consists of configuring all sensors and motors and driving forwards until an object is detected. From colour sensor input information, the robot will identify whether the object is an obstacle or food bowl. Depending on which, the robot will either move past the obstacle or dispense a certain preprogrammed amount of food correlating to the information retrieved from the sensor into the bowl. The robot will continue this routine until all bowls have been filled and will return back to its starting position where it undergoes a shut-down procedure until its routine is initialized again by the owner. For this project demonstration, a number of two bowls of colours blue and purple was programmed into the robot. The blue bowl was to contain approximately 1 cup of food while the purple bowl was to contain approximately 3 cups [1].

2.1 Measurements and Detection

In order for the robot to complete its tasks appropriately, it requires measurements and inputs from the owner and its environment. The first input that the robot must receive to start its routine is through the Touch LED sensor. The owner must press the Touch LED sensor for the robot to begin its configuration and start its routine.

In order for the robot to know when to dispense food, it must receive information from the bumper sensor. The bumper is positioned on a lever arm in such a way that when the robot drives forward, the bumper will come into contact with the object that lies beside the robot's path. When the bumper is being pressed, it signals to the robot that it is beside the object, it must stop driving forward, and the colour of the object is ready to be detected.

The robot must know whether the bumper detected a food bowl or an obstacle. If an obstacle, the robot must move past it without dispensing food. If a food bowl, the robot must be able to dispense a controlled portion into the bowl. To do this, it must receive information from the optical sensor. Once the robot has detected an object beside it, the optical sensor detects the colour of the object. If the colour does not correspond to any of the colours of the bowls, the robot will see the object as an obstacle and must raise the bumper arm to move past it without dispensing food.

The bumper arm will be lowered once the colour of the obstacle is no longer detected. If the optical sensor detects a colour equal to a colour of one of the bowls, the robot will be ready to dispense food. This colour will correspond to a preprogrammed portion of food to be dispensed into the bowl. After the food has been dispensed, the robot must raise the lever arm containing the bumper and drive forward until the colour of the bowl is no longer detected. After the robot has passed the bowl, the lever arm can be relowered for the next bowl to be detected.

For the robot to have the correct orientation and drive forward at 90 degrees, a gyro sensor must be used to calibrate the robot.

2.2 Motors

The robot will move throughout its environment in order to complete its tasks. It will utilize two motors attached to the front wheels to drive forward beside the bowls, stop when an object is detected by the bumper sensor, and drive backwards to its starting position when it has completed its task.

One motor will be used to control how much food is dispensed into the bowl. The motor will be attached to a bar-like structure, initially positioned to keep the trapdoor for the food container closed. Once the robot is aligned with the food bowl and the colour has been detected, the motor will activate and move the bar away from the trapdoor, allowing food to flow out. The motor will only move the bar away for the preprogrammed amount of time that corresponds with the colour of the bowl detected by the optical sensor. Once the time runs out, the motor will move the bar back into its initial position, closing the trapdoor and not allowing any more food to flow out.

One motor will be used to control the raising and lowering of the lever arm containing the bumper sensor. The robot initially starts with the lever arm upright to save space. Once the Touch LED is pressed, the motor attached to the lever arm will rotate 90 degrees so that the arm is parallel with the ground. As the robot drives forward and the bumper comes into contact with an object, the robot will either dispense the pet food or move past the object depending on the colour detected. Afterwards, the motor rotates to its initial orientation, raising the lever arm back to its upright position. This position is kept while the robot drives forward until the colour of the object is no longer detected. Then, the motor will again turn 90 degrees for the lever arm to be parallel to the ground and the process is repeated until all bowls are filled. Once the robot has completed its task, the motor raises the bumper arm to its upright position before traveling back to its starting location to prevent the arm from hitting any objects on the way back.

2.3 Completion and Shutdown

There will be a pre-programmed amount of bowls that the robot must fill with pet food. Once the robot has filled that certain number of bowls, it will initiate the shutdown procedure, beginning with raising the bumper arm and travelling backwards to its starting position. Once it has arrived to its initial location, the robot's driving motors are stopped.

2.4 Changes

In the earlier stages of this project, the robot would only be able to dispense a fixed amount of food into each bowl. To make the robot more useful, its routine was changed by adding the colour detection portion that would allow for differing amounts of food to be dispensed in different coloured bowls.

Initially, the robot's starting position was to be far away from the bowls and the robot was to rotate 360 degrees until the color of the bowl was detected. This method would not have worked due to the limitations of the optical sensor and the impracticalness of the design.

Another change was that the robot was going to dispense food from its front rather than its side. This method of dispensing food would have been difficult to achieve as this would have involved the robot needing to maneuver around the bowls precisely for the food to be dispensed in the proper place. This makes it impractical for the owner and relies on unrealistic circumstances that the bowls are always the same distance apart.

3.0 Constraints and Criteria

3.1 Constraints

For the first constraint, the bowls must be aligned within a defined area so that when the robot travels in a straight line, it remains approximately 3-5 inches away from the bowl. This allows the lever arm with the bumper sensor on the robot to reliably reach the bowl to signal to the robot when to stop and dispense the food (when the bumper presses against the bowl, and the colour sensor detects the colour of the bowl).

Changes: This constraint was added after our informal presentation. Previously, the design did not feature a lever arm and instead had a self-aligning mechanism on the front to guide the robot. This was since changed to feature the new lever arm design. Hence, this constraint was added. Additionally, it must be contained within this area so that the colour sensor can accurately sense the hue of the robot so it dispenses the right amount of food.

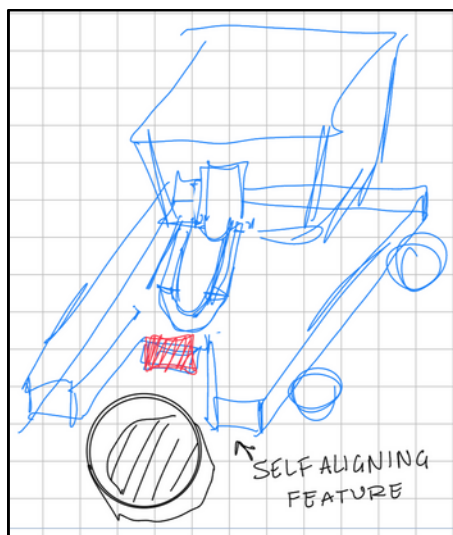


Figure 1: Original robot self aligning mechanism sketch

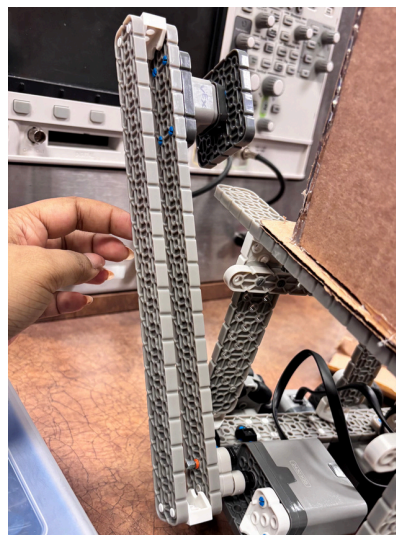


Figure 2: New bumper lever design

Another constraint is that the robot must be compact enough to be easily stored in a home environment. This is important for aesthetic purposes in a home. To address this constraint, the code was modified so that the lever arm lifts up to be positioned perpendicular to the ground before returning to its home position. This ensures that the robot can be compactly stored in the users designated storage spot. Lifting the lever arm ensures that the robot can fit within a 1.5ft x 1.5ft x 1.5ft dimension. The team also prioritized a compact form by incorporating an easily removable food container, giving the user the option to store the container separately.

Changes: The previous design did not include the lever arm mechanism, and instead had the self-aligning mechanism as pictured above. This previous design would not have fit within the desired 1.5ft x 1.5ft x 1.5ft dimension and would not be as compact as the new lever arm design.

The last constraint was that the robot must allow for easy refilling and cleaning. To meet this constraint, a container with a removable lid on the top was selected. This allows for easy food refilling. Additionally, the container is easily detachable from the main robot in order for easy cleaning.

Changes: Previously, the robot was designed so that the container would be permanently attached to the robot. However, to simplify the cleaning/refilling process, a 3D printed a base was added instead to hold the food container. This allowed for the container to be easily removable.

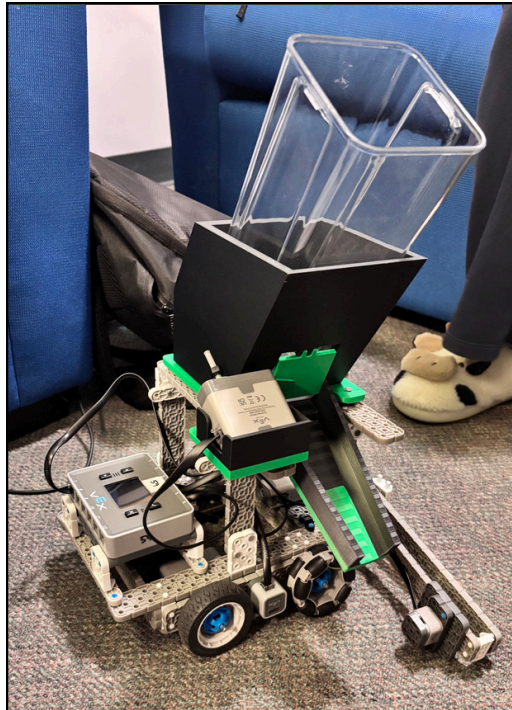


Figure 3: 3D printed base with container on robot

3.2 Criteria

For the first criteria, the robot is required to dispense the correct amount of food into the bowl depending on the bowl colour. To meet this criteria, the code for the trapdoor mechanism (in green, shown above) was calibrated to be open for different amounts of time depending on the colour of the bowl. (purple--the larger bowl, and blue--the smaller bowl).

Changes: Previously, this criteria was only to dispense food into one bowl. However, it was then decided that this idea was too simple, so the criteria was changed so that the robot dispenses different amounts of food depending on the bowl colour.

For the second criteria, the robot must dispense the food at the correct time. To accomplish this, a lever arm was created using the VEX Lego with a bumper sensor being attached to the end of it. It will extend down so that it is parallel to the floor (similar to a barrier gate for car parking), and will stop moving forward and dispense the food when the bumper on the arm touches the bowl.

Changes: Instead of having a guiding mechanism at the front to align the robot, with the food dispensing on the front, the food will dispense from the side and the lever will prompt the robot to dispense the food at the right time.

3.3 Criteria and Constraint implementation

Overall, all of the criteria and constraints have helped in deciding the best design for the robot. From the criteria, the robot needed to dispense food at the correct time and in the correct amount depending on the bowl colour. To do this, the code was changed so that the trapdoor was opened for different amounts of time depending on the hue that the optical sensor detected (purple or blue), and the lever arm with the bumper sensor was added to the design to aid in the timing of the dispensing to let the robot know when to stop.

From the constraints, the robot needed to be close enough to the bowl for the optical sensor to detect the hue of the bowl properly and self align. So, the lever arm was added. Additionally, the robot was constrained to be small enough to be stored away and the container needed to be easily removable for cleaning. Hence, the lever arm is lifted while the robot moves back home and there was a 3D printed base to hold the container.

4.0 Mechanical Design and Implementation

The design of the robot utilizes a chassis and motor drive system similar to the original configuration of the robot, a lever arm, a container holder and base, and a trapdoor mechanism. It also features the bumper, optical, and touch sensor. The design of this robot prioritizes simplicity in its design to streamline 3D printing, uncomplex mechanisms and features to simplify the code needed to run them, and an unobtrusive perimeter to allow the robot to be out of the way when not in use. The final design utilized the VEX Lego parts, and SolidWorks to design the custom pieces which were 3D printed in PLA.

4.1 Chassis and Motor Drive

The design of the robot base, consisting of the chassis and motor drive system, is relatively similar to the original configurations of the VEX robot. The main function of the base is to support the container in its size and weight, and let the wheels sit in a configuration which will allow the robot to drive forward and backward easily. Therefore the width and length of the chassis were extended, and the VEX brain was situated at the head of the robot to give enough room for the chosen container's dimensions (3.5in x 3.5in x 9in) to fit within the centre of mass, so it doesn't tip over if the robot moves too quickly. Next, supports were placed in the remaining space on the robot, built up to hold the container in the centre of the robot. These supports were made high enough so the container design and slide would have enough room to use gravity to its advantage on the slide, and be able to reach out of the perimeter of the chassis to where the dog bowls would be.

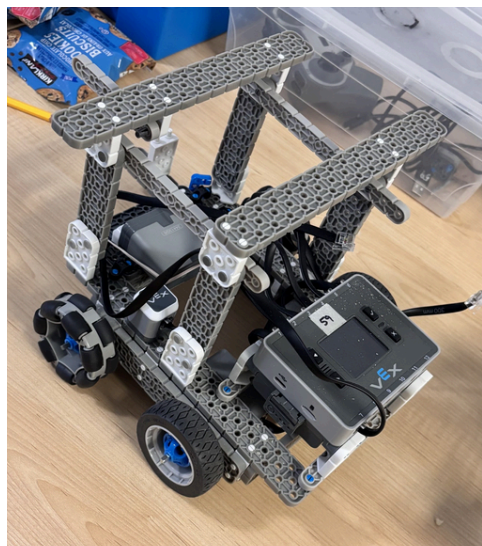


Figure 4: Robot chassis and supports

4.2 Lever

A lever arm is used on the back of the robot chassis to house the bumper out of the robot's perimeter to make contact with the dog bowls when the robot was in the proper place to deposit the food. The lever arm was built using long Lego pieces, doubled up in width so it had more strength when put up against the dog bowls. The longest Lego pieces available were used to construct the lever because the bumper must hit in the rough center of the dog bowls side so there is no slippage and it gets fully pressed. There needed to be enough room between the outermost edge of the robot chassis and the radius of the bowl so they did not connect accidentally. The bumper was placed on the very end, sideways, so the wire was not sticking straight up or down. It was attached to a motor which was placed under the container supports, facing the back of the robot. Enough testing found that when the bumper made contact with the bowl, the lever would bend backwards instead of staying perpendicular with the robot. This caused the bumper to never be pressed by the bowl which skipped the rest of the actions the robot was supposed to complete. This was remedied by the addition of a support behind where the bumper sat when it was lowered completely. This support stopped the arm from being pushed backwards when it hit the dog bowls, allowing the bumper to be pressed and the appropriate code to run. The motor turning the lever was given extra support using a rubber band to make sure it did not lift off the chassis when the lever arm hit an obstacle.

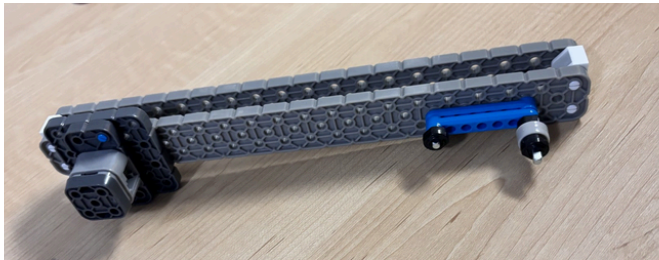


Figure 5: Lever arm

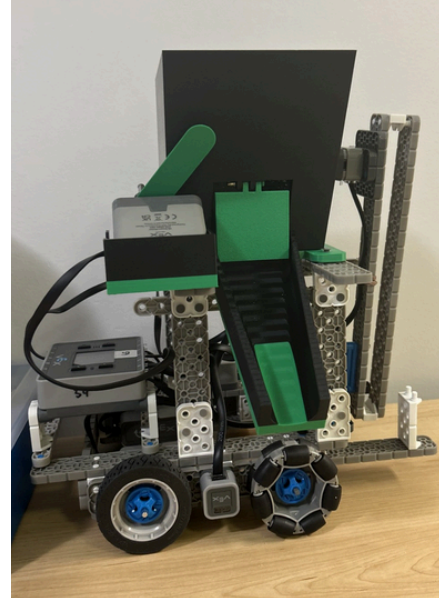


Figure 6: Lever, pushback support

4.3 Sensor Attachment

The sensors used in this robot were the bumper, optical, and touch sensor. The bumper was attached to the end of the lever arm as stated in the previous subsection. The optical sensor was placed on the left side of the robot facing where the slide and trapdoor will open to. It was attached low on the chassis in between the wheels so it would be low enough to read the colour of the dog bowls, and in line with the slide so when the robot stops to deposit the food, the sensor can read the colour. The touch sensor was placed on the right side of the robot, under the supports for the food container. The touch sensor needed to be placed somewhere easy to reach so it was on the outer edge of the perimeter without anything in the way.

4.4 Container Holder and Slide

The food container which was purchased at an outside retailer, needed to be held up at the top of the robot so food could come out of it down a chute to the dog bowls. Therefore, a holder was designed on SolidWorks to keep this container in place, attached firmly to the robot base. The base of the holder was made in context with online downloaded VEX Lego parts to match up hole placements and sizes. Originally, the container was to sit on top of the supports in a 3D printed holder parallel to the supports and chassis, however after further design iterations and conversations, the bottom of the holder was designed to have a 30 degree incline. This incline sat the container at an angle which allowed gravity to assist in pushing the food out of the square opening carved in the side of the container. The holder was designed with a 3.75 in² hole through the top, drafted upwards at 1 degree to accommodate the outward draft of the original container. There was a rough 10mm width on all sides of the walls so they don't crack under any potential force from the container moving, and a 2in x 2in opening through the front face of the holder to match the 2in x 2in opening cut out of the container. A slide was added starting at the square opening of the holder, which extends roughly 3.5in outside of the perimeter of the robot to account for the distance the dog bowls will need to be so they read on the colour sensor but still do not come in contact with the robot chassis or wheels. This slide, while it starts at the same width as the opening, lofts to a width of 1in to help funnel the dog food into a more controlled flow size. This makes sure the food is directed to the rough centre of both bowl sizes and none of the food will miss the bowl when it is flowing out.

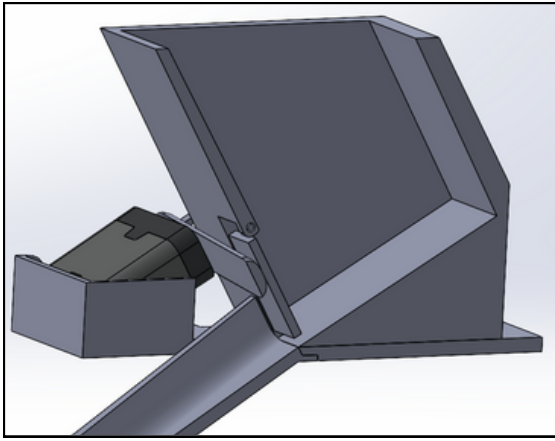


Figure 7: Section view of CAD, internal incline

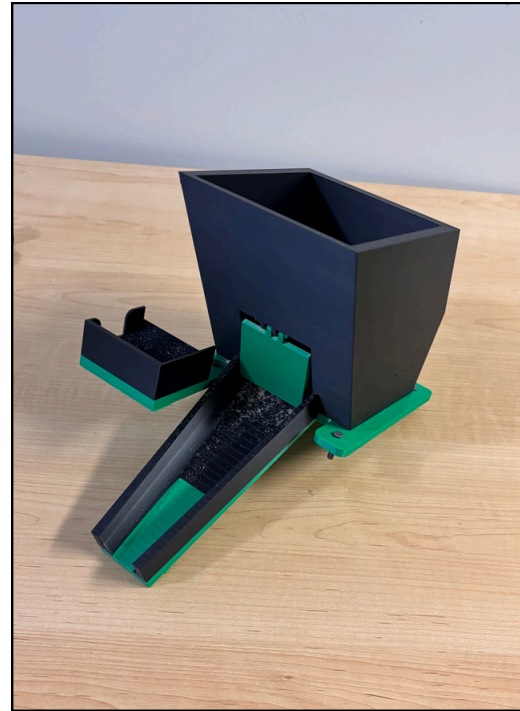


Figure 8: 3D printed base

4.5 Trapdoor Mechanism

The trapdoor was the most heavily reiterated mechanism. Throughout the design process, its placement, mechanics, and resting orientation was edited continuously until a design was found that upheld all the most important needs and restrictions. Initially, the trapdoor was positioned on the bottom of the container, which would allow for gravity to push the food out when the door slid out of place. Then, the trapdoor was positioned facing the front of the robot with a slide to bring the food to the bowls, because the way the robot approached the bowls changed to pulling up in front of them. This iteration had the trapdoor hinge open using a motor shaft as the pin of the hinge to motorize the door open and closed. This design had some flaws in the use of the motorized hinge, as it was hard to orient the motor against the container holder in a way that it did not interfere with the holder while still being close enough to sit within the hinge. Once the incline was added to the container holder, a new solution for the trapdoor emerged using gravity to its advantage.

Since the front face of the container holder would be at an angle, the trapdoor would need to be at that angle when it's closed, however, the door's normal configuration would be perpendicular to the ground because of gravity, leaving the hole in the container open, letting food out. The door was given a tolerance of 0.06in from the opening of the holder, to allow it to swing freely and not have any interference between the holder's opening or the slide width. The motor was fitted with a switch perpendicular to the a shaft which would hold the trapdoor closed, and rotate to let the door swing open to its natural positioning. The motor was placed in an angled holder, matching the angle of the front face of the holder to avoid any interference when it turns the switch. The switch was made with large fillets on its

edges so it would glide along the door when forcing it closed. The door and hinge was made to sit within the face of the container holder when closed so the switch on the motor would not hit the hinge when sliding over the door or meet any other kind of interference which would stop it from closing the trapdoor. All together, the switch, trapdoor, and hinge pin would be 3D printed.

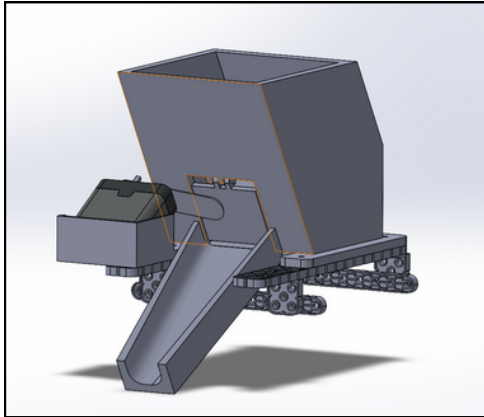


Figure 9: CAD of entire trapdoor mechanism

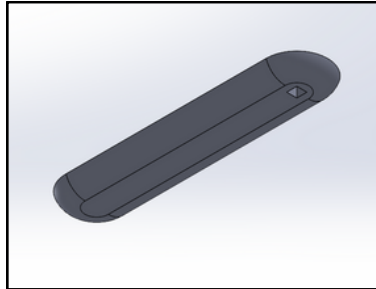


Figure 10: CAD of switch

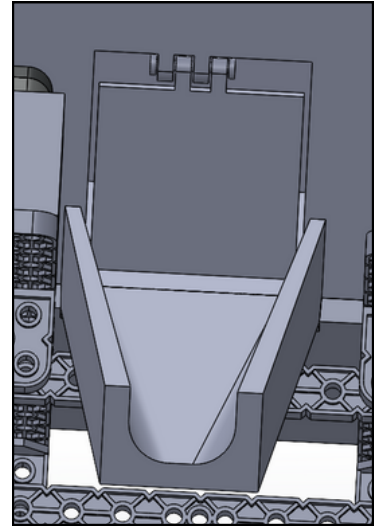


Figure 11: CAD of trapdoor and slide

4.6 Printing and Overall Assembly

All of the SolidWorks designed parts were 3D printed at the Printing Office in E7 using the Bambu Lab printers and green and black PLA. The printing process showed some struggles as the prints kept failing since it was a very large assembly to print. To combat this, some larger parts of the assembly were split to be printed separately and glued together later. The black container holder was printed separate from the green base which attaches it to the VEX parts and slide. The slide was split from the base with a lip to allow the slide to be glued to the base in the exact orientation it was intended with more surface area for the glue to stick to, giving a stronger fastening point. Printing the slide separate from the base was important to optimize printing time and filament usage, as the slide would have needed a large amount of support material in every potential printing configuration. Our base pieces and slide were printed with split colours because the printer ran out of filament half way through the printing time, and was reloaded with a different colour PLA, however this colour combination adds to the aesthetics of the design. Once the prints finally ran through properly, after about three days worth of attempting to print, the pieces were ready to be assembled with the robot chassis and VEX supports. The container fit perfectly in the designated space in the holder, and since the printed holder base was designed in context to the VEX pieces, all of the holes matched up with the VEX support base, allowing for it to be pieced together using the Lego pins and fasteners. All the separated pieces were glued together and the trapdoor mechanism was able to be tested and refined.

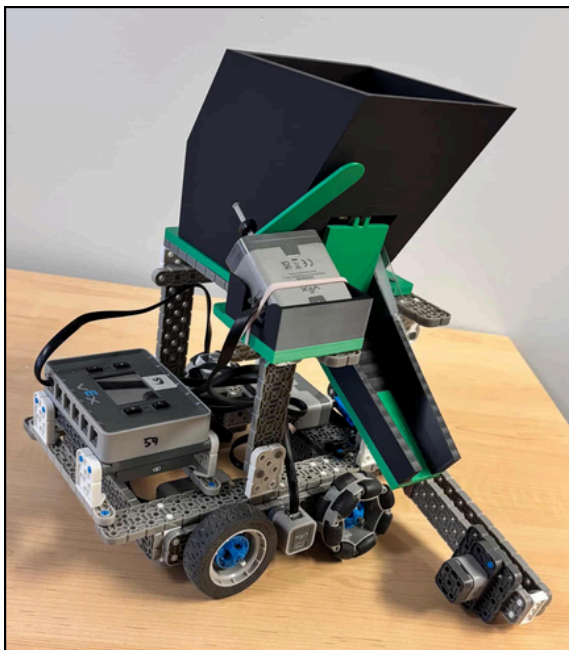


Figure 12: Complete 3D print assembled

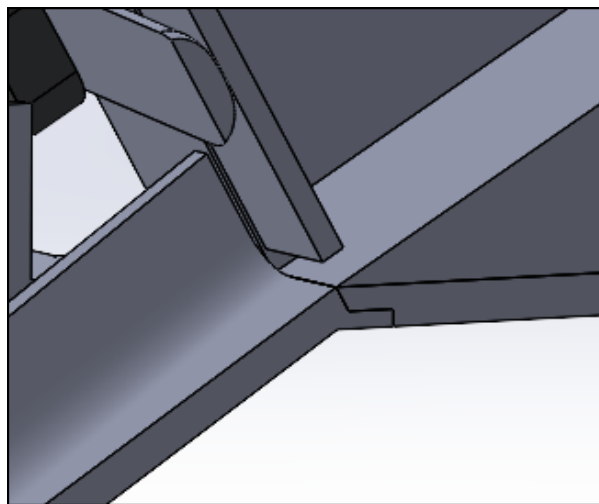


Figure 13: Section view of slide lip lead in

5.0 Software Design and Implementation

The software design process for the dog food dispensing robot focused on creating efficient, testable code capable of receiving input from multiple different sensors, and used those values to output to actuators.

5.1 Software Design Overview

Program Structure and Task Breakdown

The overall program functionality is broken down into multiple independent, sequential tasks, that have one designated goal. Each task the robot completed, corresponds to a contained function in the code. This design was chosen for its modularity and the simplicity it introduced to the testing process. The robot's behaviour was divided into logical blocks based on what actions needed to happen at each stage of the routine. Low-level action and sensing tasks, such as detecting a bowl's color and driving until the bumper is pressed or dispensing food, were all placed in individual functions. This allowed them to be modified and tested separately without affecting the rest of the program. The main control logic for the feeding cycle, was grouped into a single function, which called the smaller functions in the correct order. Finally, the setup and initialization steps were all stored in a single function. This separates configuration from action. This structure ensures that each portion of the robot's routine is isolated, making it easy to test each individual portion and more efficient to update.

Task List and Changes

Table 1: Final Task List for Demonstration and Notes on Changes

Task Category	Task Description	Previous Iteration	Reason for Change
Management/Testing	Ensure testing occurs in a well-lit area	Bowls are unsecured and lighting is variable	To improve consistency with the sensor outputs, leading to more accurate results
	Tape dog bowls to the floor		
System Start-up	Ready message is displayed on screen	Ready message is not displayed on screen	Added confirmation to ensure sensors are fully calibrated before user starts the robot
Unexpected Event	The robot does not dispense food when an object with incorrect hue is detected	N/A	N/A

Table 1 (continued)

Shut Down Procedure	Robot reverses back to home position	The lever is not raised to home position	To emphasize the compact design and real-world usability
	Robot drive motors are stopped		
	Lever arm is raised to resting position		
System Operation	Lever arm is lowered	N/A	N/A
	Robot drives forward in a straight line		
	Robot stops when bumper is pressed against the bowl		
	Trap door is opened by the motor and the correct amount of food is dispensed		
	Lever arm is raised		
	Robot moves forward past the bowl		
	Lever arm is lowered again		
Shut Down Procedure	Robot reverses back to home position	The lever is not raised to home position	To emphasize the compact design and real-world usability
	Robot drive motors are stopped		
	Lever arm is raised to resting position		

Function descriptions

The primary functions used to implement the system are detailed below, with the complete source code available in Appendix A.

Table 2: Key Program Functions and Implementation Details

Function Name	Author	Parameters and Return Type	Description
main	Rithanya	Int main()	Initializes VEXcode, configures sensors, and starts the main routine. Stops program when done. See Figure 14.
moveHome	Rithanya	void moveHome()	Returns the robot to its starting position and results the bumper arm. See Figure 15.
configureAllSensors	This function was provided in MTE 121	void configureAllSensors()	Initializes and calibrates the inertial sensor, resets motor positions, and sets up the Brain screen.
startRoutine	Rithanya	void startRoutine()	Runs the full feeding routine: waits for start signal, raises bumper, loops through bowls to detect color, dispense food, and adjust position. Ends by moving home. See Figure 16.
moveUntilBumper	Maria	void moveUntilBumper()	Drives the robot forward until the bumper touches a bowl, then stops. See Figure 17.

Table 2 (continued)

detectColor	Waffa	double detectColor()	Reads hue from the optical sensor and returns a dispensing time based on the bowl color. See Figure 18.
releaseFood	Mishal	void releaseFood(double time)	Dispenses food from the trapdoor for a duration based on the detected bowl colour. See Figure 19.
afterDetection	Rithanya	void afterDetection()	Adjusts the robot's position after detecting a bowl and moves forward until the next bowl is detected. See Figure 20.

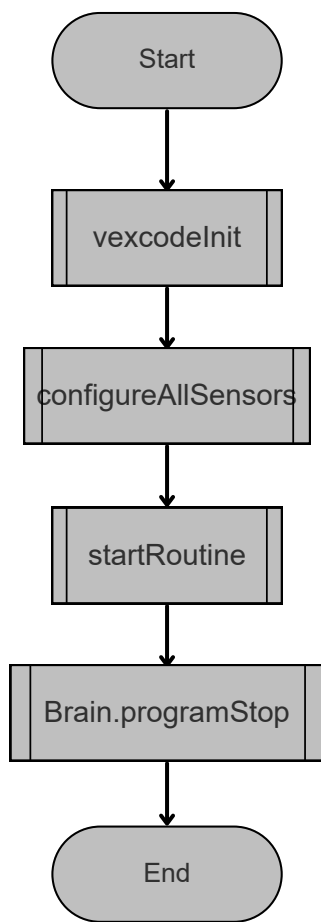


Figure 14: Flowchart for main()

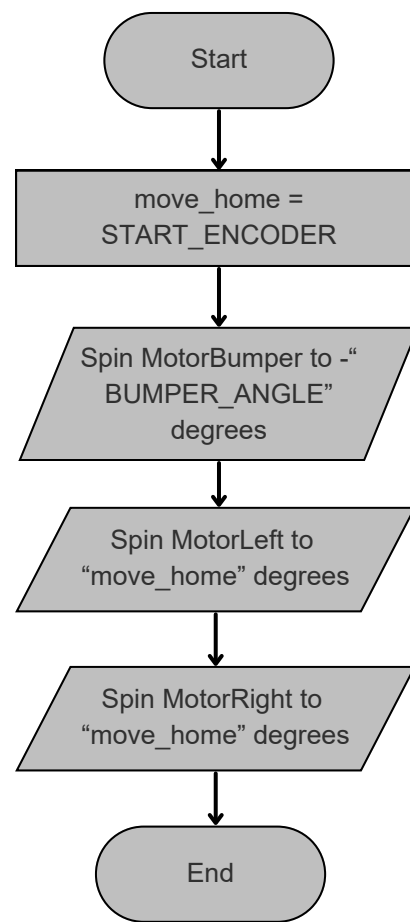


Figure 15: Flowchart for moveHome()

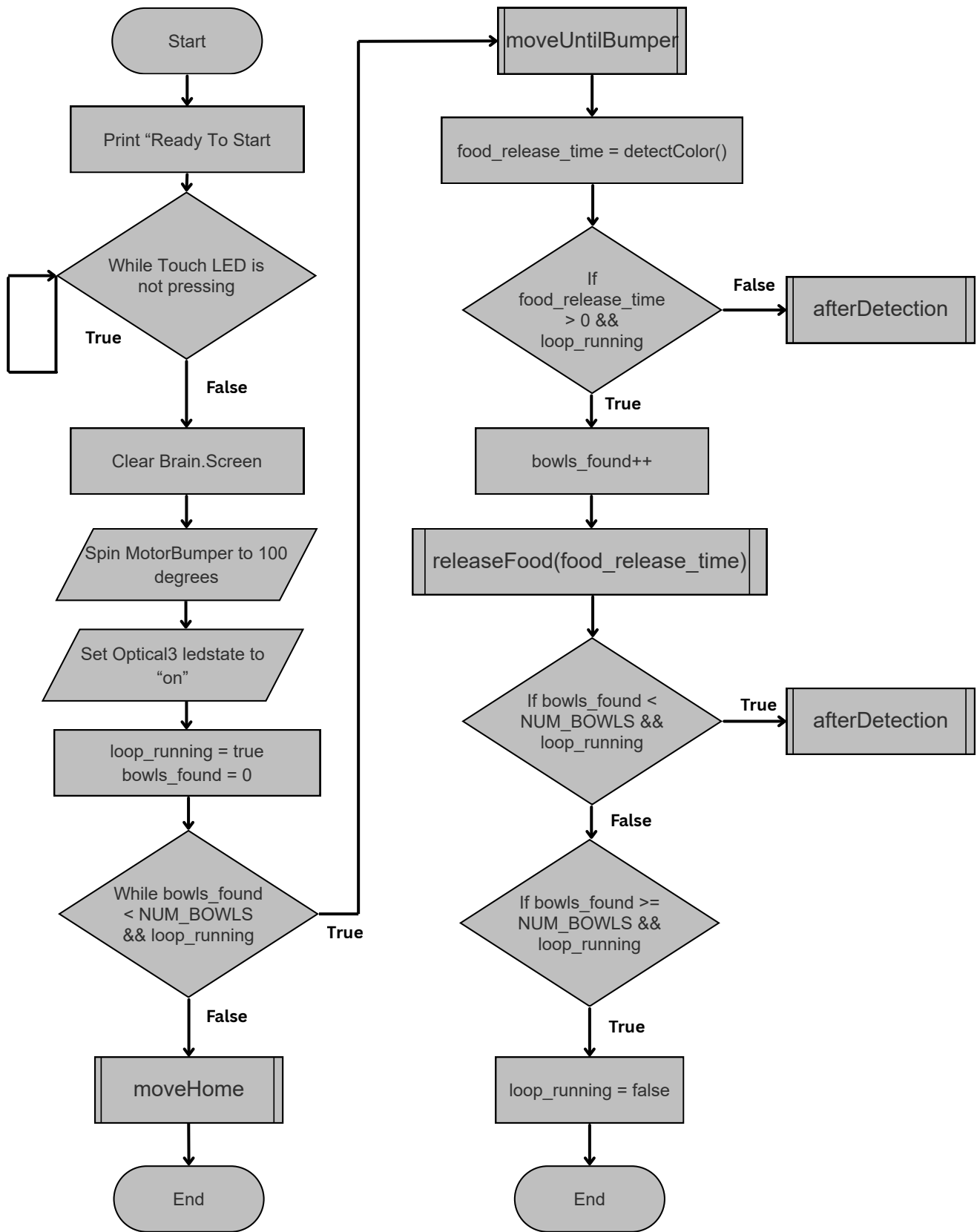


Figure 16: Flowchart for startRoutine()

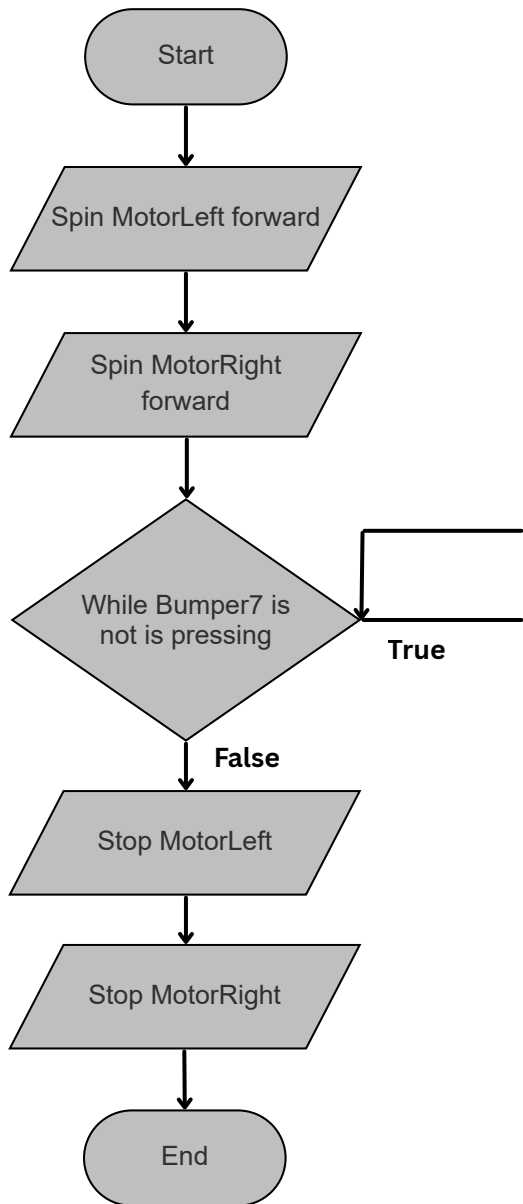


Figure 17: Flowchart for `moveUntilBumper()`

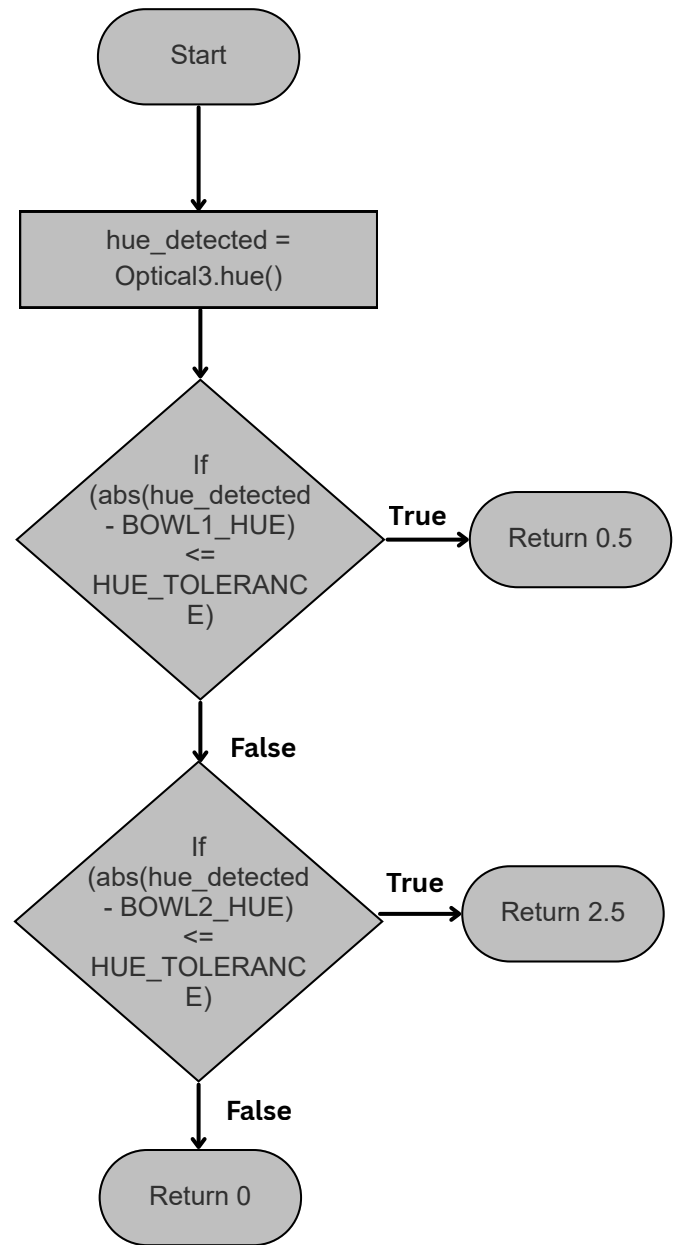


Figure 18: Flowchart for `detectColor()`

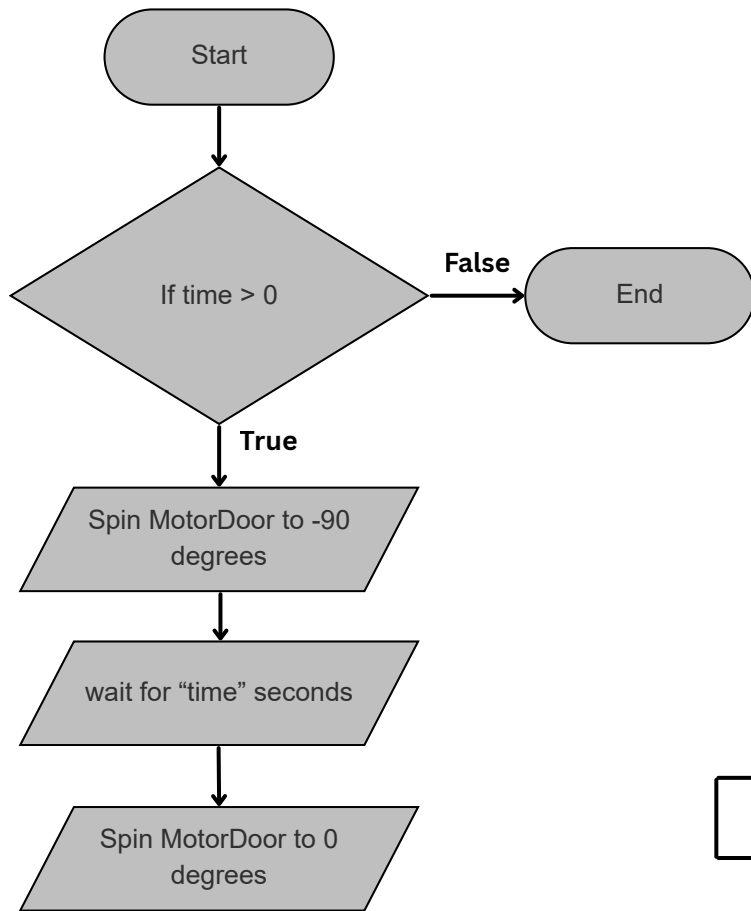


Figure 19: Flowchart for `releaseFood()`

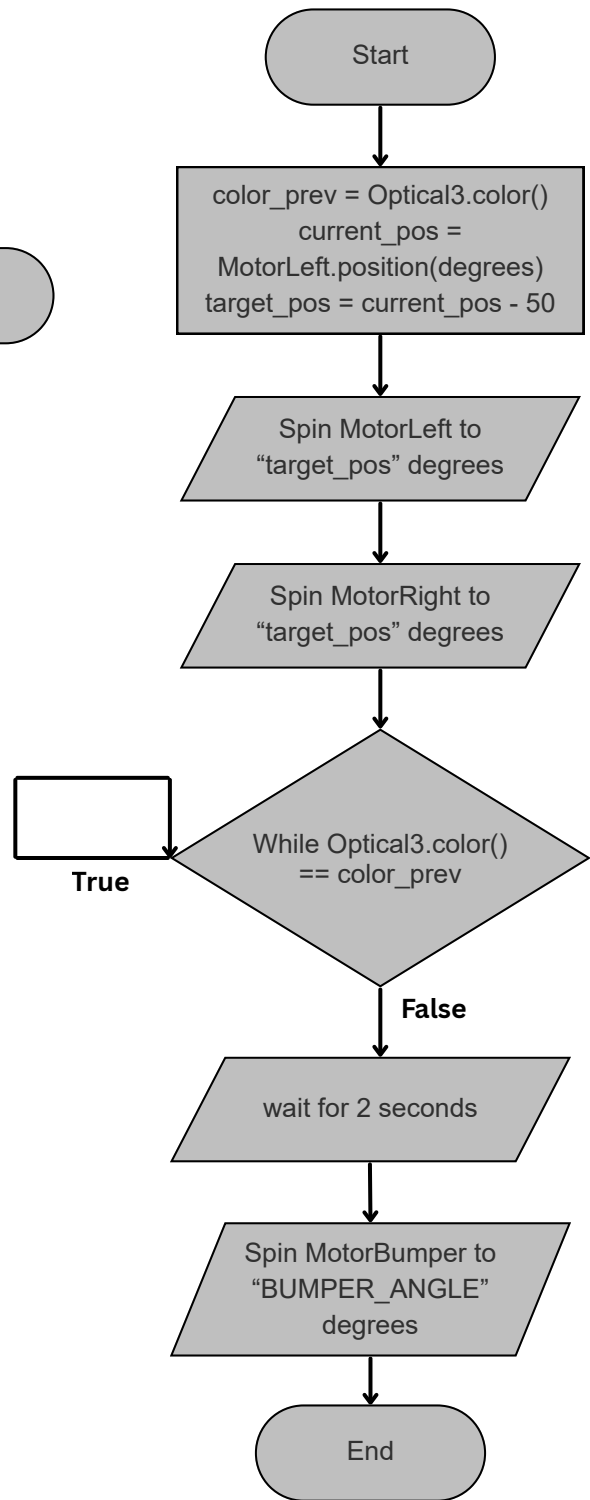


Figure 20: Flowchart for `afterDetection()`

Data Storage and Management

In this program, constants store fixed values that are used throughout the program for easy access, if any updates are necessary. Sensors (inertial, optical and bumper) store dynamic readings such as orientation, color hue, and pressed or unpressed states. These are read in real-time to control movement and actions. Motors store encoder positions, which is used by the program to move the robot and operate the robot. Lastly, local variables inside functions store constant, dynamic and temporary data. They contain data such as current motor positions, detected colors and food dispensing times. Display information is also stored on the Brain screen. The combination of data storage used, allows the robot to function efficiently and correctly.

5.2 Software Design Decisions and Trade-offs

Several software decisions were the result of practical constraints the team encountered while testing. The team initially planned to use the VEX remote for operational control. However, that required the use of callback functions, and it was explicitly stated that callback functions could not be used. Instead, a touch sensor was integrated and programmed as the on/off button for the robot. For colour detection, hue values were used instead of the built-in color reading because the color output was inconsistent and unreliable under different lighting conditions. Another compromise was the method used to re-position the robot after identifying the bowl. Although calculating the encoder counts needed to safely pass the bowl would have been more precise, the team did not have enough time to measure and test the correct encoder values. As a result, a timer was used as a simpler alternative, causing the robot to be less precise. Lastly, the main feeding cycle was designed to run based on real-time sensor detections rather than a fixed number of cycles. This ensured that the robot would continue searching for the correct number of bowls, regardless of the amount of unexpected objects it ran into. This design choice increased the complexity of the control logic, but provided increased reliability and flexibility during the robot demonstration.

5.3 Program Testing and Validation

Throughout the development of the software, testing was continuously performed to ensure correct behavior at every stage. Each function was tested individually, before being integrated into the main routine. This ensured that the sensor readings, motor outputs and timing-based actions were working as expected. After the individual tests, full cycle tests were conducted, with various environment changes such as using different bowl arrangements, lighting conditions, and unexpected objects, to ensure the robot behaved consistently in a variety of environments. The testing data is summarized in the table below.

Table 3: Test and Verification Summary for Program Functions

Feature	Task Performed	Purpose	Expected Behaviour	Validation Method
Sensor Configuration	Ensure testing occurs in a well-lit area	Bowls are unsecured and lighting is variable	To improve consistency with the sensor outputs, leading to more accurate results	Printed the sensor readings and motor positions to the screen, confirming they were reset
Colour Detection	Test with bowl colours and random objects	Confirm hue-based detection is reliable	Returns 0.5 or 2.5 for current hues, and returns 0 for non-bowl objects	Printed the return values from the function and ensured they corresponded with the object tested
Food Release	Run with 0.5 and 2.5 inputs	Verify door movement and timing accuracy	Door opens to -90°, holds for correct duration, then closes	Observing the motor movement, and manually timing the door's opening and closing time.
Bumper Stop	Press the bumper multiple times	Ensure robot consistently stops on contact	Both motors stop immediately when bumper is pressed	Visually check if motors have stopped, encoder readings printed to check for immediate
Back up and resume course	Trigger after the bowl detection is complete	Ensure backup distance and forward movement are correct	Robot backs up 50°, raises the arm lever, moves forward, lowers arm 2 seconds after different color is detected	Printed encoder values, and visually confirmed correct movement distance

Table 3 (Continued)

Feature	Task Performed	Purpose	Expected Behaviour	Validation Method
Home Return	Run at different starting distances and periods of time	Verify that the robot returns to the exact same start position	Motors drive to starting position, bumper resets its angle	Printed encoder readings, and compared it to the starting position, physically marked the starting and ending position,
Touch Start	Press TouchLED multiple times	Confirm program only starts with user input	Program waits until sensor is pressed	Verified that the robot did not start until TouchLED was pressed
Loop behaviour	Run full routine with 2 bowls and unexpected objects	Confirm correct bowl counting and ignoring wrong colors	Robot returns to start position after 2 bowls, ignores non-bowl objects	Sequence matched expected behaviour
Whole System Integration	Multiple runs with varied bowl spacing and lighting conditions	Ensure stability and repeatability across full routine	Performs the detect, release, move sequence without errors	Monitored full sequence, checked robot final position, repeated tests to confirm consistency

5.4 Significant Problems and Solutions

During testing and integration, a significant problem was with the main loop in `startRoutine()`. While all the individual functions worked correctly on their own, combining them into a loop to handle an unknown number of unexpected objects caused the robot to keep running indefinitely, and ignore any input from the optical sensor. To resolve this, the `loop_running` variable was introduced as part of the while loop condition. This variable was set to true while the loop should run, and switched to false when the loop needs to be exited. Additionally, nested if statements were added to handle multiple scenarios that weren't accounted for in a single if statement. This ensures that the robot correctly responded to all detection outcomes.

6.0 Verification

The following section discusses how each updated criteria and constraint was verified before the demo and during the project demo, as well as an evaluation of if the constraints and criteria were met. Additionally, any design updates required to meet the criteria during the demo and considerations for future trials are discussed.

6.1 Meeting Each Constraint and Criteria

Constraint 1: Bowl alignment within 3-5 inches

Verification method: The bowls were placed in a line, collinear to each other, approximately one foot apart. The robot was placed approximately two feet away from the bowls so that when it drove forward, it would drive parallel to the bowls within 3-5 inches away from the bowls, horizontally. To verify our constraint, the robot must meet the following criteria:

- The lever arm, when parallel to the ground, must make contact with each bowl separately
- The optical sensor must accurately detect the hue of the bowl when the lever arm touches it

Meeting this criteria would signify that the robot is properly aligned with the bowls in order for the color sensor to properly detect each bowl and the lever arm to be able to touch the bowls.

Result: For successful trials, the robot stopped when the bumper on the lever arm reached the bowl and the color sensor detected the right hue.

Constraint 2: Robot must be compact enough for storage

Verification method: To meet this constraint, the 3D printed base was designed to be compact enough that when it was placed on the VEX robot base, the robot could fit within a 1.5ft x 1.5ft x 1.5ft dimension. Additionally, during trial runs, to meet this constraint:

- The lever arm must retract back and stay in place perpendicular to the ground during the system shut down procedure when it returns back home

By meeting the above criteria, the lever arm will be held closely to the robot, ensuring that the robot fits within the 1.5ft x 1.5ft x 1.5ft dimension.

Result: For successful trials, the lever arm retracted back up to be approximately perpendicular to the ground while moving back to its original spot, ensuring the robot fits within the proper dimensions.

Constraint 3: Robot must allow for easy refilling / cleaning of the food container

Verification method: To meet this constraint, the 3D printed base was designed so that it could hold the food container in place while also being easily detachable to be cleaned.

Result: The 3D printed base allowed for the container to be held while being easily removable in under 5 seconds. Additionally, the container has a lid on top, making it easily refillable .

6.2 Modified / Unmet Constraints and Criteria

Due to mechanical errors, some constraints had to be removed and some had to be modified for the final demo in order to be met.

The first constraint listed on the previous page was that the bowls must be aligned collinear to each other so that the robot would drive in a straight line, 3-5 inches away from the bowls horizontally. However, the robot did not always move in a perfectly straight line due to uneven surfaces and the motion of the lever arm causing the robot to slightly deviate from its trajectory. To address this issue, our team had to ensure that the bowls are in the correct path of the robot by moving them around instead of having them fixed in straight line, since the robot did not always move in a straight path. The bowls had to be manually moved so that it lined up to be 3-5 inches away from the robot.

A criteria that was not fully met during the demo was that the robot had to accurately detect the hue of the bowl, and if it detected something other than the bowl, consider it an obstacle and move on in order to find either of the bowls. During the demo, however, the robot occasionally detected the bowls as an obstacle, due to the colour sensor not correctly detecting the hue of the bowls. This did not meet one of the criteria, which was to dispense food depending on the bowl colour, and since the correct bowl colour was not detected, the robot did not dispense the food. To address this issue, the bowl had to be moved during the demo so that it could be redetected as the robot continued on its path, until it properly sensed the bowl again. To fix this issue for future trials, the hue tolerance can be increased in the code for the robot, and the trials should be done in a brighter lighting so the colour of the bowls are accurately sensed.

There were not any extra constraints in the original plan that the robot did not perform during the demo. There were only additional constraints added, and some constraints that had to be modified during and before the demo.

7.0 Project Plan

7.1 Division of Tasks Among Group Members

Table 4: Work Distribution

Design	Coding	SolidWorks/3D Printing	Testing	Assembly
Mishal, Waffa, Maria, Rithanya	Rithanya (lead), Mishal, Waffa, Maria	Maria (lead), Rithanya, Waffa, Mishal	Mishal, Waffa, Maria, Rithanya	Mishal, Waffa, Maria, Rithanya

As shown above, tasks were distributed equally, with leads for the coding and 3D printing category. Each member contributed for the designing, coding, 3D printing, testing, and assembly, while the leads for coding and 3D printing took charge of ensuring the functionality of the code and SolidWorks design. As for the project plan, an initial Gantt chart was created before the formal presentation to outline the expected timings and due dates for the project:

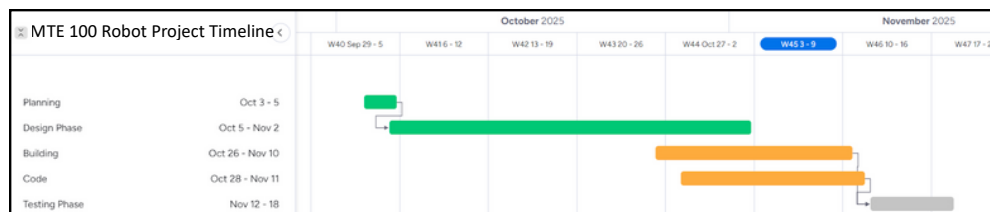


Figure 21: Original Gantt chart for project timeline

Due to several mechanical design changes, testing errors, and 3D printing errors, the project timeline was changed. The design phase lasted longer than initially expected, due to mechanical redesigns of the robot. In the chart above, the coding timeline ranged from October 28 - November 11. However, the code was actually modified from around October 11 - November 20. Additionally, the robot was tested for each change that occurred between October 15 - November 20. The 3D printing took longer than planned (and was not included in the original Gantt chart) due to issues with available 3D printers, and the print failing twice from not having enough supports for the print.

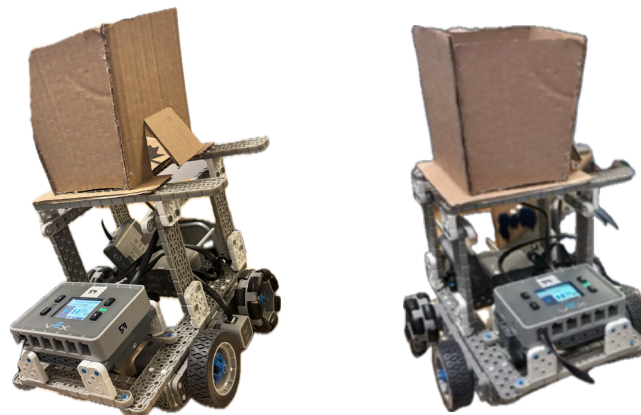


Figure 22: Two angles of the cardboard prototype for the 3D printed base

As part of the project planning, the above figures show the cardboard prototype made which was later 3D printed. This was done for the formal presentation in order to describe what our robot was and how it would look later on. The cardboard base was made with close to accurate measurements of the actual 3D print.

8.0

Conclusions

The final objective of this project was to create an automated pet-feeding robot that could dispense certain portions of food into multiple bowls and return to its starting position using provided VEX components and 3D printed pieces. Throughout the entire design and manufacturing process, changes needed to be made in structural designs and software logic. In the end, the robot was able to fulfill its line of tasks consisting of configuring the sensors and motors, driving beside the food bowls, delivering visibly consistent portions into each bowl, and driving back to its starting position.

Although 8/10 of the demonstrations using finalized mechanical structures and software were able to accurately complete the objective, 2 of the trials encountered food getting jammed in the trapdoor, causing flow of food to stop before the desired amount of time and not allowing the trapdoor to be properly closed. This created minor inconsistencies in the portioning of food that was dispensed. This also created issues when the robot began to drive forward after dispensing food. The movement of the robot sometimes dislodged the trapdoor, causing some food to flow out before the robot reached the next bowl. This became extremely inconvenient to clean up and reset.

An impracticality of the design was that the bowls had to be lined up and the robot had to be well-positioned beside the bowls in order for the bumper arm to make contact with them. During the testing phase this had proven to be difficult. If the robot was even slightly misaligned with the line-up of bowls, it would veer off into a slight diagonal route, causing the bumper arm to be unable to make contact with any of the bowls. This small variance in the set-up of the demonstration required the robot needing to constantly be reset so that it precisely aligned with the bowls.

Another impracticality stemmed from the colour detection portion of the robot's function. In different lightings, the hues of the bowls registered differently, causing the robot to identify the food bowls as an obstacle and move past it without dispensing food.

9.0 Recommendations

9.1 Mechanical Design Recommendation

9.11 Performance and Reliability: Trapdoor

The robot did not always function perfectly and a large contributor to this was certain design elements. An aspect of the robot's performance which could be improved upon in the future would be the trapdoor mechanism design. Currently, this aspect of the robot is not as reliable as it should be with it working near perfectly roughly eight out of ten times, and getting jammed and not working two out of ten times. Because this is arguably the most important mechanism of the robot as it delivers the main aspect of the robot, getting food into the dog bowls, it needs to work every single time. There are different ways of fixing this mechanism, one of which could be a complete redesign of how it works, opting for potentially a guillotine type of door with vertical tracks instead of a hinge. This change would fix the problem of the door getting jammed with food when it is trying to swing back closed, because the door would instead be pushed down with more force using a motor attached to gears and a belt to run the door up and down, though, this fix would need testing to prove it capable of not getting jammed in a similar way as the current design.

Another solution could simply iterate on the current hinged door design and focus on exactly where it falls short. The trapdoor is currently letting food through the tolerances between the door and the square opening of the holder, so shrinking those tolerances would help with the jamming. As well as changing the size of the slide closer to the trapdoor, as it is slightly too small that it can hit the door when it swings out, holding it in place which adds to the jamming. The slide's walls would also need to be raised closer to where it meets the main container holder as food will spill out the sides of the slides because of the sheer amount of food leaving the door at once.

9.12 Suitability For the Home: Materials and Detachability

An aspect separate from the performance of the robot which could be improved upon is the robot's functionality and suitability for the home. The end goal of this robot is to be used in people's homes without much need for supervision or extra work, as this robot is supposed to help make feeding pets easier. Therefore the slide and trapdoor could be redesigned in the future to give an added ease in cleaning and reattaching for the homeowner. Currently, the trapdoor and slide are permanently attached to the robot, even though they touch the food coming out. This is not ideal because they would need to be cleaned every now and then to keep up food safety and cleanliness. A fix to this would be to have the slide easily detachable from the robot to be washed. The trapdoor should then be attached to the plastic container, not the container holder, so it can be washed with the container since it would not be so easily detachable otherwise.

Another factor of suitability for the home is the material with which the slide and trapdoor are made. Since they are currently made of PLA, these components are not dishwasher safe [2] which would make cleaning them harder, and not always 100% food safe which could harm the pets. Therefore, using a material like plastic would fix these problems, leading to a safer robot for the pets and a more convenient robot for the pet owner.

9.2 Software Design Recommendation

If more time were available, the software could be improved by enhancing the color detection, for example, by averaging the hues to reduce misreads. The timed movements in both the `afterDetection` and `releaseFood` functions should be replaced with more precise methods such as using encoder counts and load measurements to improve consistency and accuracy. For industry applications, implementing error handling functions such as timeouts or sensor fault detection, would improve reliability. Adding a remote-controlled or automated start function would also increase versatility. Overall, these changes would enhance the accuracy and reliability of the robot.

10.0 Back Matter

References

[1] Jennifer Coates, “How Much to Feed Your Dog,” pet MD, September 19, 2025
<https://www.petmd.com/dog/nutrition/are-you-feeding-your-dog-right-amount>

[2] Elliot Johnson Hall, “Is PLA Food Safe?,” All3DP.PRO, February 7, 2024,
<https://all3dp.com/2/is-pla-food-safe-what-you-really-need-to-know/>

Appendix A - C++ Code

```
#pragma region VEXcode Generated Robot Configuration
// Make sure all required headers are included.
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <math.h>
#include <string.h>

#include "vex.h"

using namespace vex;

// Brain should be defined by default
brain Brain;

// START IQ MACROS
#define waitUntil(condition)
\
\  do {
\
\    wait(5, msec);
\
\  } while (!(condition))
#define repeat(iterations)
\
\  for (int iterator = 0; iterator < iterations; iterator++)
// END IQ MACROS

// Robot configuration code.
inertial BrainInertial = inertial();
motor MotorLeft = motor(PORT6, true);
motor MotorRight = motor(PORT1, false);
optical Optical3 = optical(PORT3);
motor MotorDoor = motor(PORT5, false);
motor MotorBumper = motor(PORT2, false);
touchled TouchLED4 = touchled(PORT4);
bumper Bumper7 = bumper(PORT7);

// generating and setting random seed
void initializeRandomSeed(){
  wait(100,msec);
  double xAxis = BrainInertial.acceleration(xaxis) * 1000;
  double yAxis = BrainInertial.acceleration(yaxis) * 1000;
  double zAxis = BrainInertial.acceleration(zaxis) * 1000;
  // Combine these values into a single integer
  int seed = int(
    xAxis + yAxis + zAxis
  );
  // Set the seed
  srand(seed);
}
```

```

// Converts a color to a string
const char* convertColorToString(color col) {
    if (col == colorType::red) return "red";
    else if (col == colorType::green) return "green";
    else if (col == colorType::blue) return "blue";
    else if (col == colorType::white) return "white";
    else if (col == colorType::yellow) return "yellow";
    else if (col == colorType::orange) return "orange";
    else if (col == colorType::purple) return "purple";
    else if (col == colorType::cyan) return "cyan";
    else if (col == colorType::black) return "black";
    else if (col == colorType::transparent) return "transparent";
    else if (col == colorType::red_violet) return "red_violet";
    else if (col == colorType::violet) return "violet";
    else if (col == colorType::blue_violet) return "blue_violet";
    else if (col == colorType::blue_green) return "blue_green";
    else if (col == colorType::yellow_green) return "yellow_green";
    else if (col == colorType::yellow_orange) return "yellow_orange";
    else if (col == colorType::red_orange) return "red_orange";
    else if (col == colorType::none) return "none";
    else return "unknown";
}

```

```

// Convert colorType to string
const char* convertColorToString(colorType col) {
    if (col == colorType::red) return "red";
    else if (col == colorType::green) return "green";
    else if (col == colorType::blue) return "blue";
    else if (col == colorType::white) return "white";
    else if (col == colorType::yellow) return "yellow";
    else if (col == colorType::orange) return "orange";
    else if (col == colorType::purple) return "purple";
    else if (col == colorType::cyan) return "cyan";
    else if (col == colorType::black) return "black";
    else if (col == colorType::transparent) return "transparent";
    else if (col == colorType::red_violet) return "red_violet";
    else if (col == colorType::violet) return "violet";
    else if (col == colorType::blue_violet) return "blue_violet";
    else if (col == colorType::blue_green) return "blue_green";
    else if (col == colorType::yellow_green) return "yellow_green";
    else if (col == colorType::yellow_orange) return "yellow_orange";
    else if (col == colorType::red_orange) return "red_orange";
    else if (col == colorType::none) return "none";
    else return "unknown";
}

```

```

void vexcodeInit() {
    // Initializing random seed.
    initializeRandomSeed();
}

```

```

#pragma endregion VEXcode Generated Robot Configuration

```

```

#include "iq_cpp.h"
using namespace vex;

```

```

const int STANDARD_SPEED = 30;
const double START_ENCODER = 0;
const int HUE_TOLERANCE = 15;
const int BUMPER_ANGLE = 90;

const int NUM_BOWLS = 2;
const int BOWL1_HUE = 220;
const int BOWL2_HUE = 310;

void configureAllSensors(){
  BrainInertial.calibrate();
  wait(2,seconds);
  BrainInertial.setHeading(0,degrees);
  BrainInertial.setRotation(0,degrees);
  MotorLeft.setPosition(0,turns);
  MotorRight.setPosition(0,turns);
  Brain.Screen.clearScreen();
  Brain.Screen.setFont(mono15);
}

void releaseFood(double time)
{
  if (time > 0)
  {
    MotorDoor.spinToPosition(-90,degrees);
    wait(time, seconds);
    MotorDoor.spinToPosition(0,degrees);
  }
}

double detectColor()
{
  int hue_detected = Optical3.hue();
  if ((abs(hue_detected - BOWL1_HUE) <= HUE_TOLERANCE))
  {
    return 0.5;
  }
  else if ((abs(hue_detected - BOWL2_HUE) <= HUE_TOLERANCE))
  {
    return 2.5;
  }
  else
  {
    return 0;
  }
}

void moveUntilBumper()
{
  MotorLeft.spin(forward, STANDARD_SPEED, percent);
  MotorRight.spin(forward, STANDARD_SPEED, percent);

  while (!Bumper7.pressing()){

    MotorLeft.stop();
    MotorRight.stop();
  }
}

```



```

void afterDetection()
{
    color color_prev = Optical3.color();
    double current_pos = MotorLeft.position(degrees);
    double target_pos = current_pos - 50;

    MotorLeft.spinToPosition(target_pos,degrees,false);
    MotorRight.spinToPosition(target_pos, degrees,false);

    MotorBumper.spinToPosition(-100,degrees, false);

    MotorLeft.spin(forward, STANDARD_SPEED, percent);
    MotorRight.spin(forward, STANDARD_SPEED, percent);

    while (Optical3.color() == color_prev){ }
    wait(2, seconds);
    MotorBumper.spinToPosition(BUMPER_ANGLE,degrees);
}

void moveHome()
{
    double move_home = START_ENCODER;
    MotorBumper.spinToPosition(-BUMPER_ANGLE, degrees, false);
    MotorLeft.spinToPosition(move_home, degrees,false);
    MotorRight.spinToPosition(move_home, degrees);
}

void startRoutine()
{
    Brain.Screen.print("Ready To Start");
    while(!TouchLED4.pressing()){ }
    Brain.Screen.clearScreen();

    MotorBumper.spinToPosition(100, degrees, false);
    Optical3.setLight(ledState::on);

    bool loop_running = true;
    int bowls_found = 0;
    while (bowls_found < NUM_BOWLS && loop_running)
    {
        moveUntilBumper();
        double food_release_time = detectColor();
        if (food_release_time > 0 && loop_running)
        {
            bowls_found++;
            releaseFood(food_release_time);

            if (bowls_found < NUM_BOWLS && loop_running)
            {
                afterDetection();
            } else if (bowls_found >= NUM_BOWLS && loop_running)
            {
                loop_running = false;
            }
        } else
        {
            afterDetection();
        }
    }
    moveHome();
}

```

```
int main()
{
    vexcodeInit();
    configureAllSensors();
    startRoutine();
    Brain.programStop();
}
```